

Lab 7

In this lab, you will review and then continue implementing the `RecursiveList` class we've been writing.

Recursive Lists

Begin by downloading a starter implementation from Google Classroom or the course webpage (<http://courses.knox.edu/cs142>) under "Assignments". The given code is packaged as a .zip file; unzip this and open the project by double clicking on `lab7.code-workspace`.

I've actually removed the method implementations that we've already written. You'll begin by recreating these; try to do it on your own before looking at the code posted online.

Start with the `size` method, empty bodies for which have already been added. Begin by providing the implementation for `EmptyNode`. Remember that this method should give the size of an empty list. Then write an implementation for `NonEmptyNode`. This version should give the size of the list starting at the node whose method is called. It will need to make a recursive call to `next.size` in order to get the size of the list starting with the next node.

Once you have those methods implemented, see if they work. A test case is in `RecursiveListTest.java`. Make sure that the test case passes.

Next, try to implement the `contains` method. (We did this in class, but see if you can do it yourself, without looking at notes or the code on the course webpage.) You'll need to uncomment the line in `Node` and write the method in `RecursiveList`. (Remember that it should just delegate all the work to the head node.) Then implement the method in `EmptyNode` (does the empty list contain the given value?). Finally, write the implementation for `NonEmptyNode`, uncomment the provided test case, and make sure it works.

As a next method, try to implement `numOccurrences`, which takes a value and returns the number of times it appears in the list. You'll need to add code in four places. I suggest you do them in the following order:

- Write a `RecursiveList` method that delegates all the work to `head`.
- Add the method signature to the `Node` interface.
- Implement the method for `EmptyNode`.
- Implement the method for `NonEmptyNode`.

Finally, write test cases and make sure that your method works.

Once `numOccurrences` works, we'll do the `get` method, for which test cases have been written. This method uses the idea that each node has its own idea about indexing. It is itself index 0, the following node is index 1, etc. When you make recursive calls in the `NonEmptyNode` implementation, you will have to subtract one from the desired index in order to switch the indexing. The method should throw an `IndexOutOfBoundsException` if the index is out of range. Again, check that your implementation works by uncommenting and running the test cases.

After getting `get` to work, add the `indexOf` method by following the same four steps that you used for `numOccurrences`. This method should take a value and return the index in the list at which that value first appears or -1 if the value doesn't appear. For this method, you'll again have to switch between the indexing schemes of different nodes.

For the rest of the lab, you'll focus on methods that change the list (adding or removing a value). The key with most of these methods is to have the recursive `Node` function return the new list. The version we did was `add` that adds to the end of the list. In `RecursiveList`, this should do

```
head = head.add(val);
```

since the `add` method of `Node` will return the new list. The `NonEmptyNode` implementation should include

```
next = next.add(val);
```

for the same reason. In addition, the `NonEmptyNode` implementation also needs to perform

```
return this;
```

since the method needs to return the list with the value added.

With this in mind, write the method. As always, begin with the method in `RecursiveList` and the signature in `Node`. Next write the implementation in `EmptyNode` by thinking what the empty list should do when asked to add to an empty list. Finally, consider the implementation for `NonEmptyNode`; again, it should return the modified list with the desired value added.

Once you have an implementation, write test cases to ensure that it works.

Next, try a version of `add` that takes a specific index at which to add the value. For example `add(0, "Hi")` would add to the front, but `add(1, "Hi")` would add after the first value. If the index is too large or small, the method should throw an `IndexOutOfBoundsException`. Implement the four parts in the order we've been using: `RecursiveList`, `Node`, `EmptyNode`, and `NonEmptyNode`. You'll also need to have the `Node` methods return the new list as we did for other methods that modify the list.

As the next variation, I suggest `addAfter`, which takes two arguments and adds the first argument to the list after the first occurrence of the second argument. If the second argument doesn't appear in the list, then `addAfter` does nothing.

Once you have `addAfter` (and know that it works through testing), a slight variation of its code should give you `addBefore`, which adds a new value immediately before the first occurrence of a given value.

If you complete those, try to write some other useful list methods: `remove(i)` (remove the value at a given index), `lastIndexOf(T)`, `set` (changes value at a given index), `replace` (replaces one value in the list with another),